CIDECODE 2026 | TEAM KAVACHA

Project LAKSHYA

Evasion-Resistant Android Malware Forensic Triage to Intercept Financial Fraud and Cluster C2 Networks.

Shreya Shenoy | Shreya Patil | Aniyath Amrita Pradeep

PES UNIVERSITY (EC CAMPUS) | BATCH OF 2024-2028

The Problem Statement

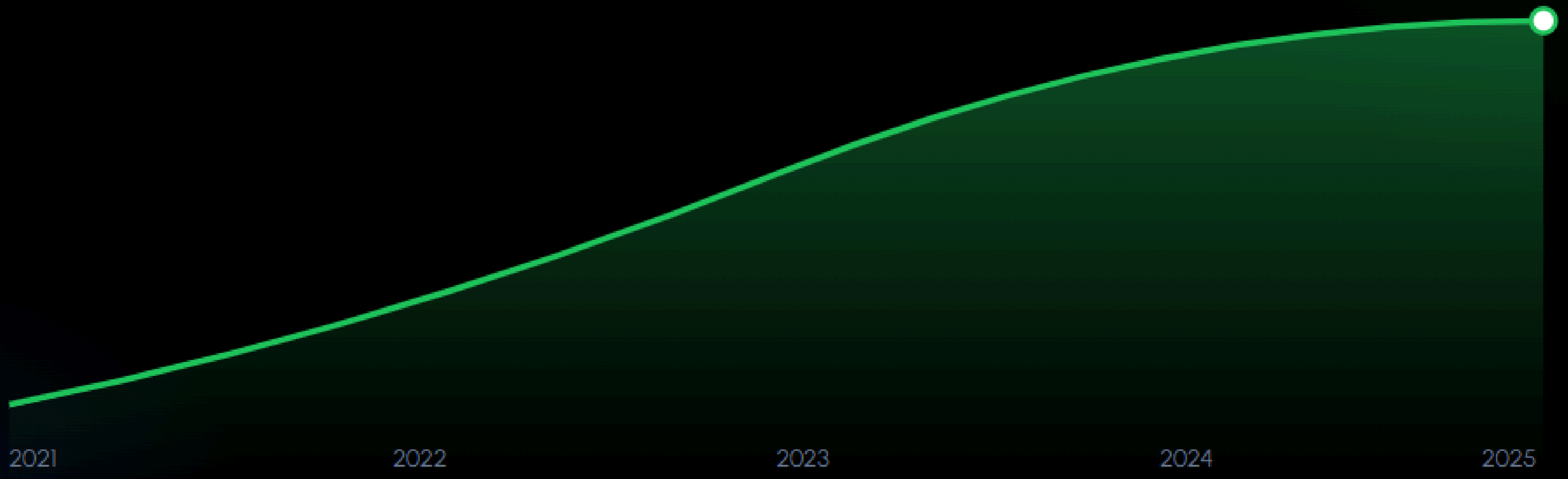
Resource Bottlenecks

Non-technical field officers lack specialized mobile forensic capabilities to rapidly triage malicious, evasion-resistant scam APKs. Current tools are often too complex for frontline station use.

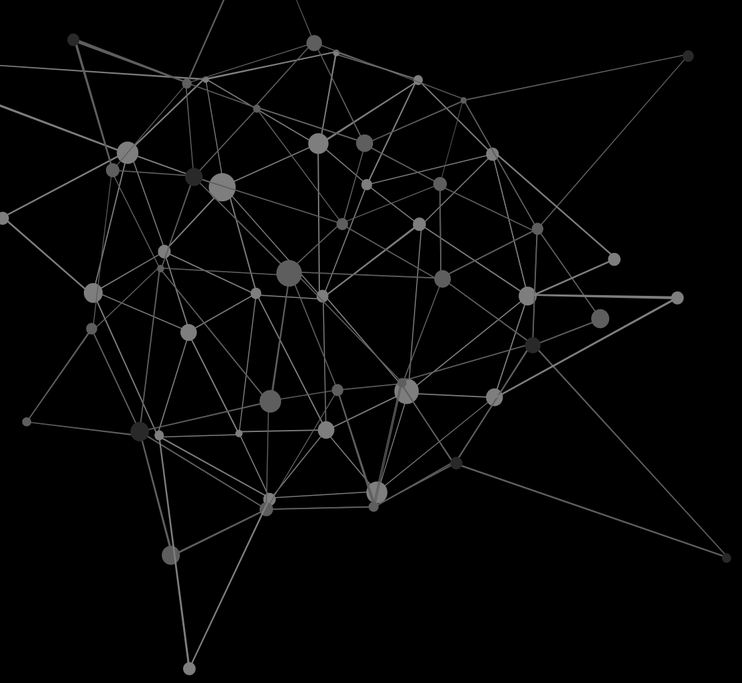
Syndicate Mapping Delays

Traditional tools evaluate seized apps as isolated incidents, failing to link recycled code variants to decentralized Command-and-Control (C2) syndicates and infrastructure.

Growth of Mobile Fraud



Projected surge in APK-based financial interceptions requiring localized triage solutions.



Introducing Project LAKSHYA

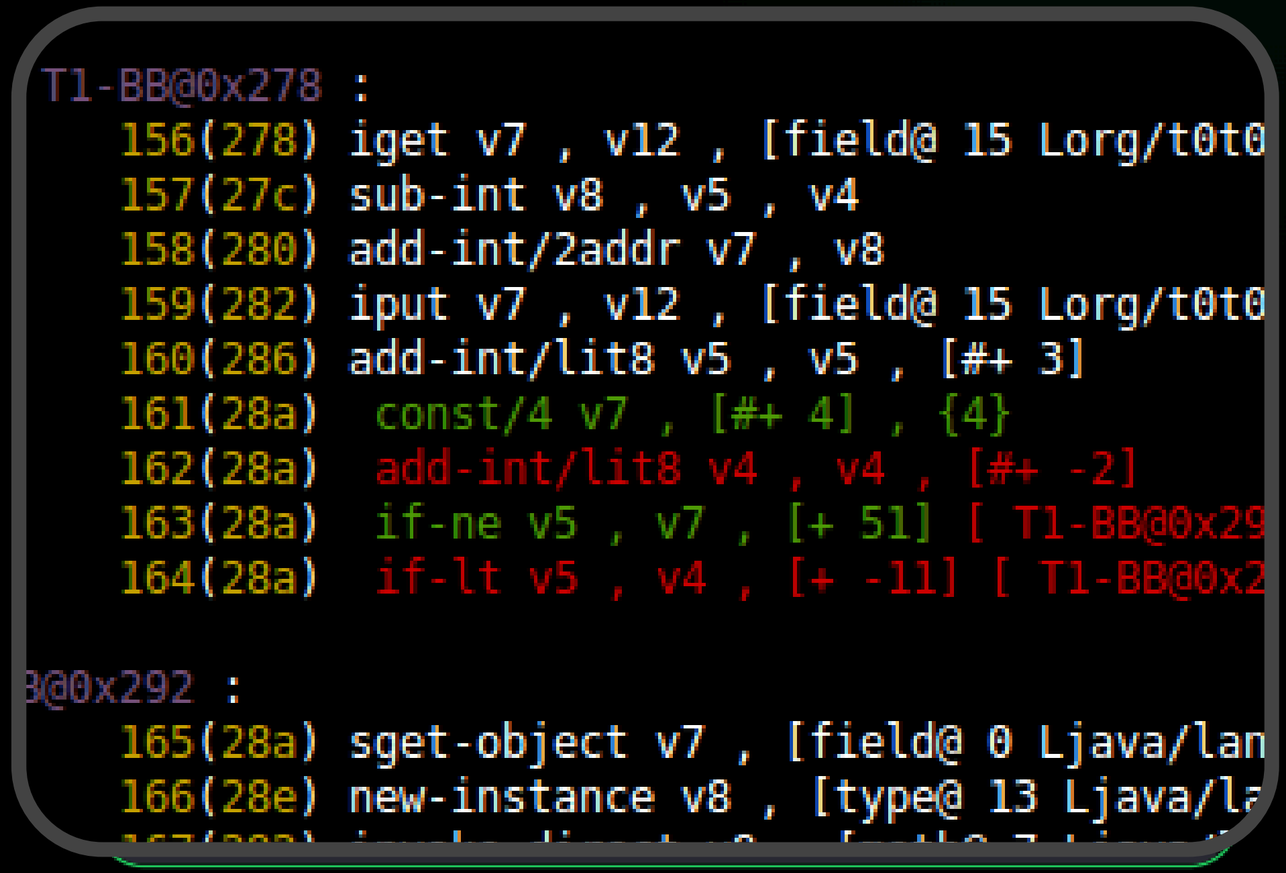
100% Evasion Resistant Static Forensics Pipeline

Traditional sandboxes fail when advanced trojans detect emulators and freeze to hide their emulators. LAKSHYA bypasses this completely by extracting full execution paths directly from raw byte code. this ensures 100% evasion resistant triage process that captures malicious intent safely without any execution risks

Reverse Engineering Core

Module 1: Static Parsing

- **Androguard Integration:** Decompiles .dex binaries into readable bytecode without executing volatile malware.
- **Permission Mapping:** Scans AndroidManifest.xml for high-risk combos like SMS intercept + Internet.
- **UI Spoofing Detection:** Identifies brand impersonation (e.g., SBI, YONO) using regex matching against author signatures.



Precision Verification Metrics

0%

sandbox dependency

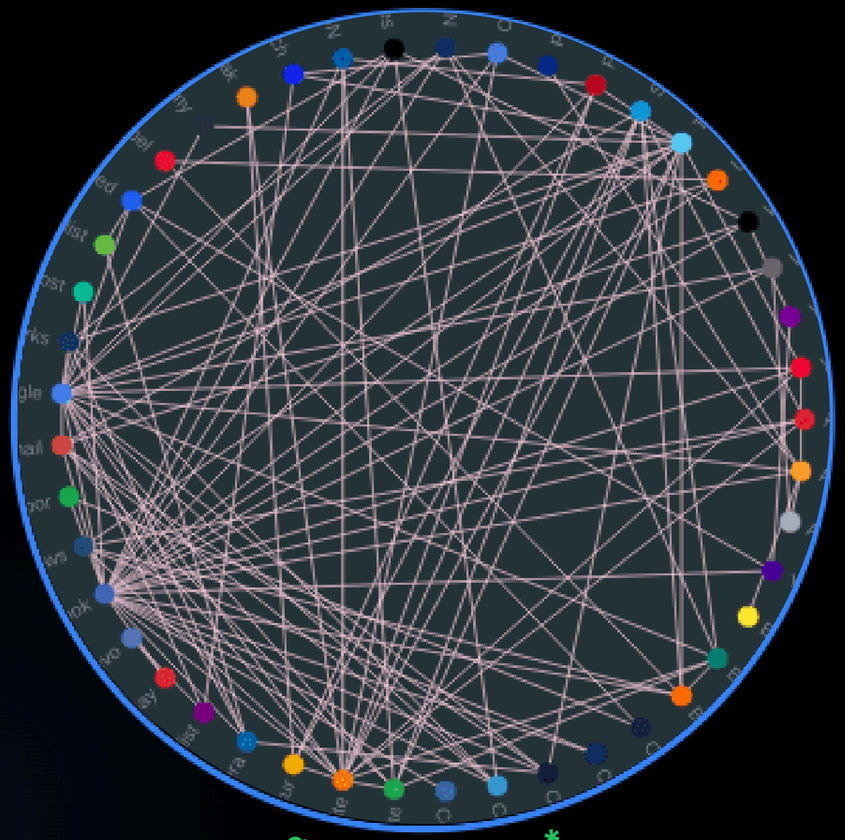
Weighted Confidence Scoring

Our heuristic engine eliminates False Positives by weighing permission risks, signature mismatches, and structural genealogy.

$$C = \sum_{i=1}^{n} w_i r_i$$

Where w is weight and r is the risk component identified.

Code DNA & Genealogy



*generic image**

Module 2: Call-Graph Fingerprinting

We treat malware like structural DNA. Using **Function Call Graphs (FCG)**, we map code interactions into unique numerical vector fingerprints.

This allows us to track "recycled" code even if a hacker changes file names or obfuscates basic variables.

Infrastructure Clustering

Module 3: C2 Intelligence

Instead of analyzing devices in isolation, we strip hardcoded URLs and IPs out of decompiled code to uncover shared server networks.

Using **NetworkX**, independent cases (e.g., Bengaluru vs. Mysuru) are clustered to a single threat actor group based on shared registrar or IP range correlation.

The Human-Centric Angle

Bilingual Dashboard

High-contrast risk badges with clear summaries in Kannada and English for frontline officers.

FIR-Ready Export

Auto-generates structured, legally sound technical summaries for immediate charge sheet attachment.

Behavior Timeline

Visualizes the sequential execution flow of malware, making it scannable for non-tech investigators.

Technical Stack & Feasibility

Layer	Technology Used	Primary Benefit
Frontend	ReactJS / Tailwind	Military-grade Dashboard & Graphs
Backend API	FastAPI / Python	High-concurrency processing
Forensic Core	Androguard / Apktool	Evasion-proof static parsing
Graph Logic	NetworkX	Multi-district C2 clustering

24-Hour Build Timeline

H 1-3

API & ingestion setup

H 3-9

Static parsing & C2 regex

H 9-12

Genealogy & Clustering

H 12-18

React UI & Localization

H 18-24

Testing & Pitch Prep